AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
INTERNATIONAL CREDIT HOURS ENGINERING PROGRAMS (ICHEP)

Hotel Management System

Milestone 1

CSE241 "Object-Oriented Programming"

Submitted to:

Dr. Mahmoud Ibrahim Khalil
Eng. Mazen Tarek Samy

Names	IDs
Abdallah Nageh Salama	25P0418
Awsam Nady Shehata	25P0419
Beshoy Hany Malak	25P0441
Hassan Abdalla Ahmed	25P0302
Sameh Amged Makram	25P0429

1. Executive Summary

This report outlines the **Hotel Management System**, a Java-based application developed to streamline hotel operations by managing room allocations, guest reservations, and financial transactions. Built with a focus on core **Object-Oriented Programming (OOP)** principles, the system ensures high scalability and modularity through the use of Inheritance, Abstraction, and Encapsulation.

The primary objectives achieved in this milestone include:

- **Architectural Foundation:** Designing a centralized data store and core domain models that serve as the backbone for all system modules.
- **Operational Workflow:** Implementing secure guest registration, dynamic reservation management, and a controlled staff interface for receptionists and administrators.
- **Financial Integrity:** Engineering a robust billing engine with automated invoice generation and custom exception handling to ensure secure and accurate payments.
- **System Reliability:** Conducting comprehensive testing across all modules to verify business logic and state transitions, ensuring a professional and bug-resistant codebase.

2. Design Decisions & System Architecture

2.1 Centralized Data Management

We opted for a **Centralized Data Store** model using the HotelDatabase class. By utilizing static repositories (ArrayLists), we ensured that data (rooms, guests, and staff) remains persistent and accessible throughout the application's lifecycle without the overhead of passing complex objects between modules.

2.2 OOP Implementation Strategy

- **Encapsulation:** We enforced strict data hiding by marking fields in classes like Invoice, Room, and Guest as private. Access is controlled through public getters and setters.
- **Inheritance & Abstraction:** The Staff class was designed as an **Abstract Class** to serve as a common blueprint. The Admin and Receptionist classes inherit shared properties while implementing their specific logic.
- **Composition:** We utilized **Composition** in the Room class, which holds a RoomType object and a List<Amenity>, decoupling the physical room's status from its categorical description.

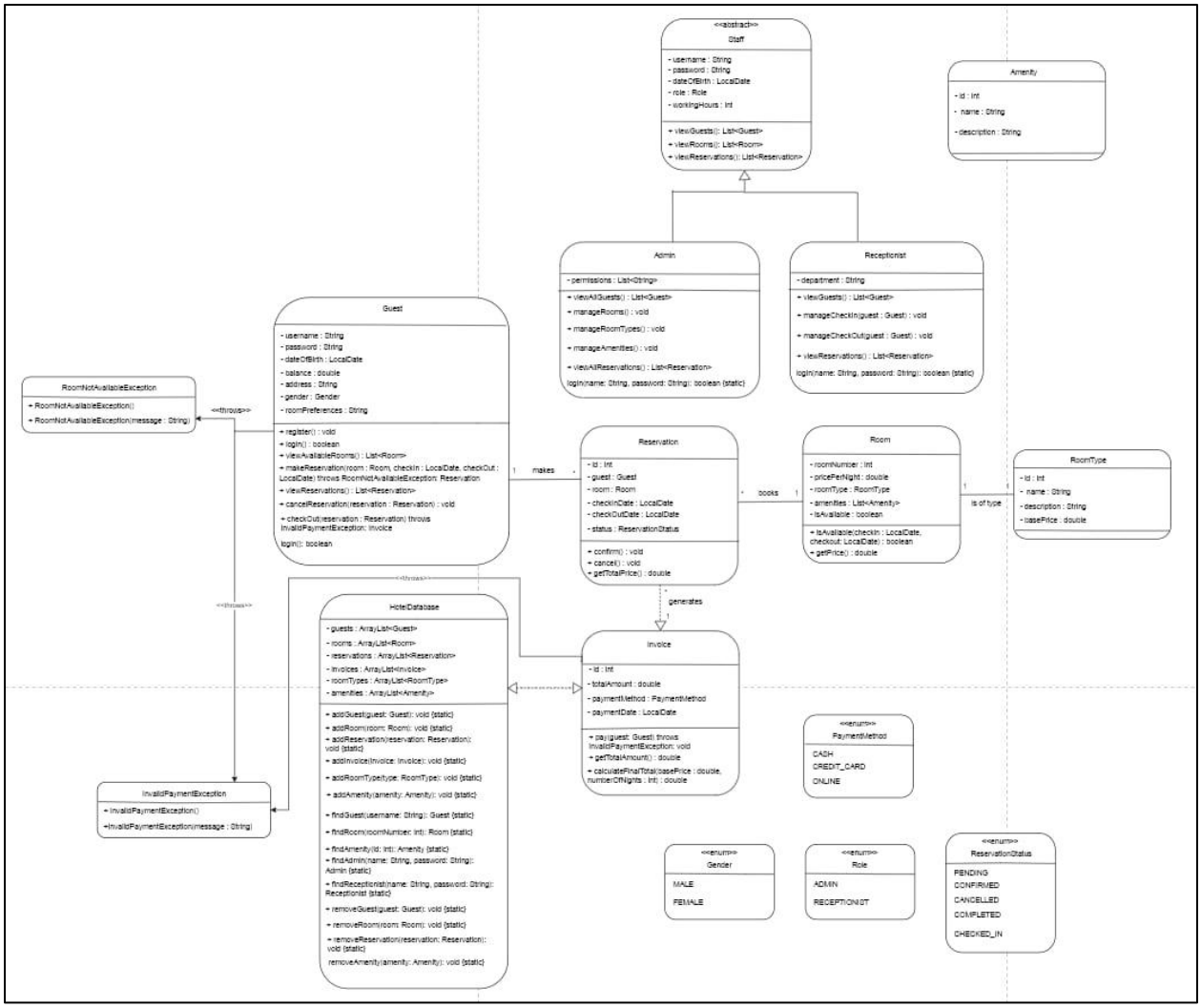
2.3 State Management & Type Safety

- **Robust Enums:** To ensure "Type Safety," we implemented Enums like ReservationStatus and RoomTypeName. This prevents bugs related to invalid string inputs.
- **Validation Logic:** We enforced a strict business lifecycle: PENDING → CONFIRMED → CHECKED_IN → COMPLETED.

2.4 Custom Exception Hierarchy

We developed a **Custom Exception Hierarchy** (e.g., InvalidPaymentException, RoomNotAvailableException). This separates business logic from error-handling code, resulting in a cleaner codebase.

3. UML Diagram



4. Detailed Contributions by Team Members

Abdallah Nageh (Lead Architect & Guest Class)

- System Foundation: Designed and implemented the entire core class architecture from scratch, creating all domain model classes including Room, RoomType, Amenity, Reservation, and Invoice.
- Centralized Data Store: Developed the HotelDatabase class as a static repository to manage all system entities (guests, rooms, staff, etc.) and implemented static methods for adding, finding, and removing data.
- Guest Module: Fully implemented the Guest class, including all business logic methods such as register(), login(), viewAvailableRooms(), and makeReservation().
- System Initializer: Wrote the setupInitialUsers() method in Main.java to seed the system with essential test data, including staff accounts, rooms, and specific guest scenarios for testing.
- Validation & Enums: Defined all five core Enum types (Gender, Role, ReservationStatus, PaymentMethod, RoomTypeName) to ensure type-safety across the platform.

Awsam Nady (Invoice Class, Room Class)

Financial Engine: Designed and implemented the Invoice class, focusing on secure payment processing and guest balance validation.

Payment Logic: Developed the pay(Guest guest) method, which verifies the guest's balance against the invoice total and throws a custom InvalidPaymentException if funds are insufficient.

Room Infrastructure: Separated physical room states from category metadata by implementing Room and RoomType classes, adhering to the principle of Composition.

Data Integrity: Implemented auto-incrementing static counters for Invoice IDs and utilized the RoomTypeName Enum to strictly define allowable room categories.

Beshoy Hany (Receptionist Class & Lifecycle Management)

Staff Hierarchy: Implemented the Receptionist class as a specialized subclass of Staff, using Inheritance to reuse common properties like authentication and roles.

Stay Lifecycle: Developed the complete manageCheckIn() and manageCheckOut() logic, ensuring that room availability is synchronized with the guest's physical stay.

Operational Guardrails: Added critical validation logic (Guard Conditions) to prevent invalid actions, such as early check-ins based on LocalDate or duplicate check-ins.

System Refactoring: Fixed critical authentication bugs by replacing reference comparisons (==) with .equals() for string-based credentials.

Hassan Abdallah (Exceptions Engineer & Reservation Class)

Reservation Engine: Built the Reservation class from scratch, implementing an auto-incrementing ID system and strict date validation in the constructor.

Financial Calculations: Developed the getTotalPrice() method using ChronoUnit.DAYS to dynamically calculate costs based on the room's price per night.

Exceptions Architect: Designed the system's custom exception hierarchy, creating RoomNotAvailableException and InvalidPaymentException to handle business logic failures gracefully.

Status Transitions: Integrated state management methods (confirm() and cancel()) and updated the checkout workflow to reset room availability upon successful completion.

Sameh Amged (Admin Class & Staff Class)

Administrative Interface: Implemented the full Admin dashboard in Main.java, allowing for authentication, viewing all guests/rooms, and managing hotel resources.

Room Management: Developed the addRoomFlow() helper method, a wizard-style interface that guides admins through creating rooms, setting prices, and assigning amenities without duplicates.

Robust Input Handling: Created private utility methods getIntInput() and getDoubleInput() to safely read user data, using validation loops to prevent system crashes from invalid inputs.

System Verification: Conducted rigorous manual testing of the Admin dashboard (10 test cases) to ensure session security and correct room deletion logic

5. Technical Specifications & Enumerations

The system relies on several Enums to maintain data integrity:

- **Role:** ADMIN, RECEPTIONIST
- **RoomTypeName:** SINGLE, DOUBLE, SUITE
- **ReservationStatus:** PENDING, CONFIRMED, CHECKED_IN, CANCELLED, COMPLETED
- **PaymentMethod:** CASH, CREDIT_CARD, ONLINE

6. Testing & Verification Summary

Test Category	Action	Result	Status
Financial	Pay with insufficient balance	Throws InvalidPaymentException	PASS
Booking	Book an occupied room	Throws RoomNotAvailableException	PASS
Admin	Add duplicate room number	"Room already exists" message	PASS
Lifecycle	Early check-in attempt	Action blocked via date validation	PASS
Core	Auto-increment ID	IDs for Invoices/ Reservations increment correctly	PASS

7. Conclusion

The Milestone 1 deliverables demonstrate a fully functional backend for the Hotel Management System. The use of advanced OOP techniques ensures that the system is ready for further UI development or database integration in future milestones.